



Proyecto Final: Sistema de Gestión de Citas ITV "Luis Vives"

Lucía Fuertes Cruz

54664615D

Índice:

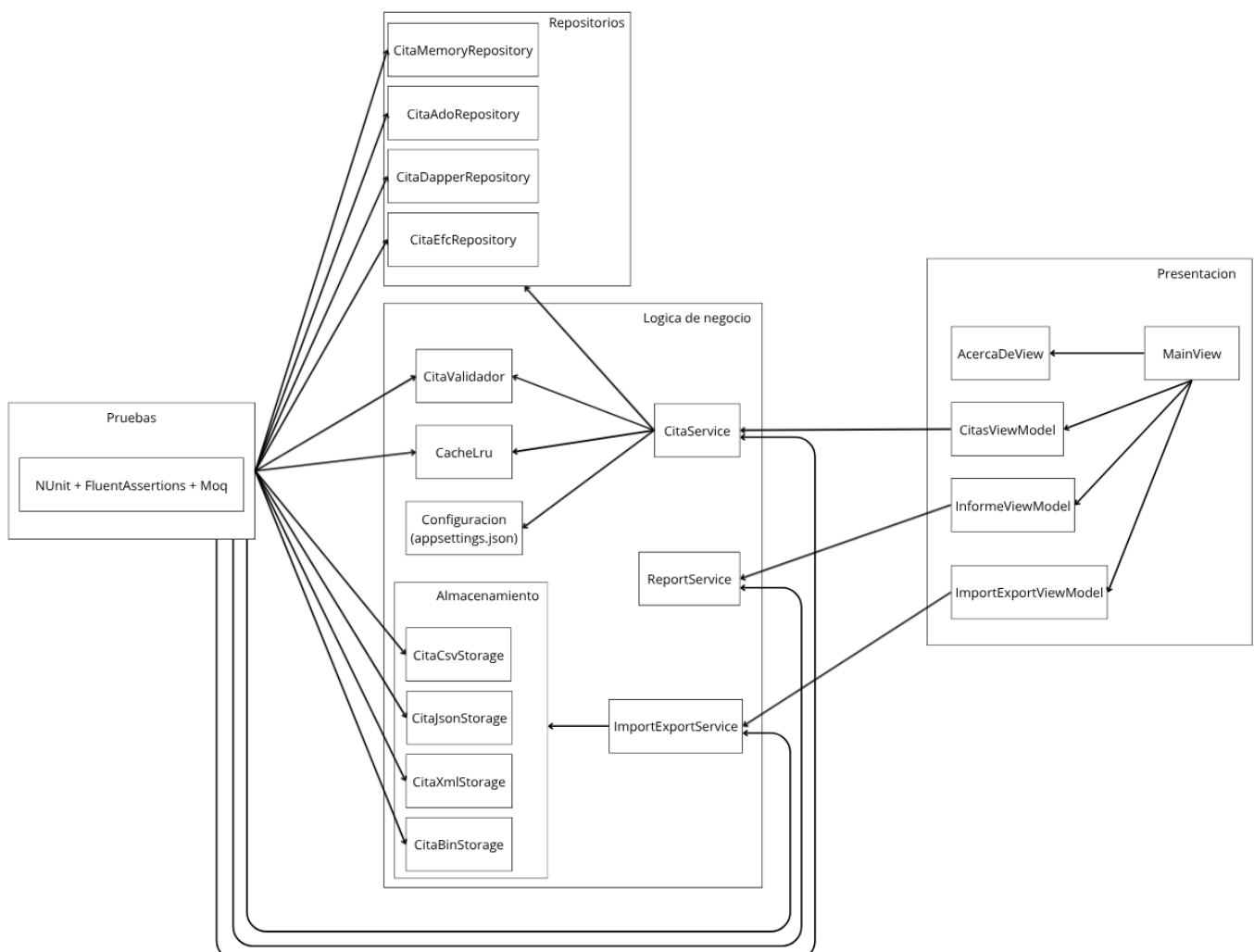
Descripción del proyecto.....	2
Requisitos funcionales.....	3
Requisitos no funcionales.....	3
Requisitos de información.....	4
Especificaciones del proyecto.....	5
Diagrama de clase.....	5
Diagrama de casos de uso.....	6
Diagrama de secuencia: Create.....	7
Diseño de bases de datos.....	10
Cobertura de pruebas.....	12
Estimacion de analisis economico.....	13

Descripción del proyecto

El sistema de gestión de citas ITV "Luis lives" está pensado con el objetivo de facilitar y ayudar a muchas estaciones de ITV que actualmente deben de gestionar de forma manual las citas que registran, estas estaciones gestionan sus citas con herramientas básicas como documentos excel.

Nuestro sistema de gestión permitirá llevar una validación de datos automática, una gestión completa del ciclo de vida de las citas permitiendo utilizar: borrados lógicos para mantener históricos, la posibilidad de almacenar la información en la propia memoria de la aplicación o de forma persistente en tres tipos de accesos a bases de datos en SQLite, generación de informes en diferentes formatos y la posibilidad de exportar e importar los datos de la aplicación en diferentes formatos, estas son algunas de las cosas que permitira nuestro sistema.

Para llevar a cabo el sistema hemos organizado un diseño de varias capas, separándolas por sus responsabilidades. Tendríamos en total tres capas, en la primera tendremos toda la lógica de negocio del sistema y los modelos, en la siguiente capa estarán todas las pruebas unitarias realizadas sobre el sistema, y en la última capa tendremos la capa de presentación en la que se ha implementado el patrón de MVVM para la interfaz del usuario.



Requisitos funcionales

Son los requisitos que indican el “que” debe de hacer el sistema, describen las funciones y acciones que la aplicación le ofrece al usuario.

- Dar de alta una cita, con validación específica para matrícula y dni, y una fecha de inspección que no supere los 30 días en el futuro.
- Modificar una cita ya registrada.
- Eliminar una cita ya registrada de forma lógica.
- Eliminar una cita ya registrada de forma permanente.
- Consultar todas las citas del sistema.
- Posibilidad de filtrado a las citas por cualquiera de sus campos.
- Posibilidad de ver citas eliminadas lógicamente.
- Importar citas en formatos: Csv, Json, Xml y Binario.
- Exportar citas en formatos: Csv, Json, Xml y Binario.
- Generar informe de las citas registradas en los formatos: Pdf y Html.

Requisitos no funcionales

Son los requisitos que indican el “cómo” debe de comportarse el sistema, especificando características de rendimiento y disponibilidad entre otras.

- Uso de Git Flow para una correcta gestión de ramas.
- Arquitectura en capas.
- Uso de cache Lru para optimización de consultas.
- Sistema de logging con Serilog por consola y mediante ficheros.
- Cobertura con pruebas unitarias superior al 80%.
- Sistema de almacenamiento con persistencia configurable en memoria, Ado.net, Dapper o Entity Framework Core.
- Sistema de exportación e importación de datos configurable en Csv, Json, Xml y Binario.

Requisitos de información

Son los requisitos que indican los datos que debe de almacenar y gestionar el sistema, definiendo la información necesaria para que el sistema funcione correctamente.

Cita:

- Id - entero generado automáticamente por las bases de datos.
- Matrícula - texto en formato español(NNNNLLL) siendo "N" un número y "L" una letra.
- Marca - texto no puede tener espacios en blanco ni estar vacío.
- Modelo - texto no puede tener espacios en blanco ni estar vacío.
- Cilindrada - entero no puede ser inferior a 0.
- Motor - enum del tipo Motor.
- Dni del dueño - texto con validación española del Dni, un Dni no puede estar en más de tres con la misma fecha de inspección.
- Fecha de matriculación - DateTime en formato español.
- Fecha de inspección - DateTime en formato español no puede ser superior a 30 días en el futuro.
- CreateAt - DateTime para el metadato de la fecha de creación.
- UpdateAt - DateTime o nulo para el metadato de la fecha de actualización si la tiene.
- IsDelete - booleano que indica si se ha borrado lógicamente esa cita.

Motor:

- Diesel
- Gasolina
- Híbrido
- Eléctrico

Informe:

- Total de citas - entero.
- Total de registros con motor diesel - entero
- Total de registros con motor gasolina - entero
- Total de registros con motor híbrido - entero
- Total de registros con motor electrico - entero
- Promedio de cilindrada del total de los registros - entero.

Especificaciones del proyecto

A lo largo del desarrollo del proyecto he ido eligiendo distintas tecnologías según lo que necesitaba en cada momento, en este apartado voy a explicar algunas de las que he utilizado y porque.

Para la interfaz he utilizado WPF en vez de WinForms por que me permite crear ventanas mucho más modernas y flexibles, sobre todo ha sido por poder usar binding y estilos. Además lo he combinado con el patrón MVVM para poder utilizar de una forma más sencilla las propiedades observables.

Por otro lado he utilizado la libreria de Serilog para el sistema de logs del proyecto, que me permite verlos tanto por consola como en un fichero, he elegido esta librería sobre el log de c# porque es más intuitiva y fácil de manejar.

En la capa que se encarga de las pruebas unitarias he utilizado Fluent Assertions para hacer los test, porque hace que las comprobaciones de los test sean más legibles y fáciles de entender que con las básicas.

Y para no tener que depender de las excepciones he decidido usar la librería de Result, esta me permite devolver errores de forma mucho más controlada y sin tener un “peligro” a causa de lanzar una excepción no controlada.

Diagrama de clase

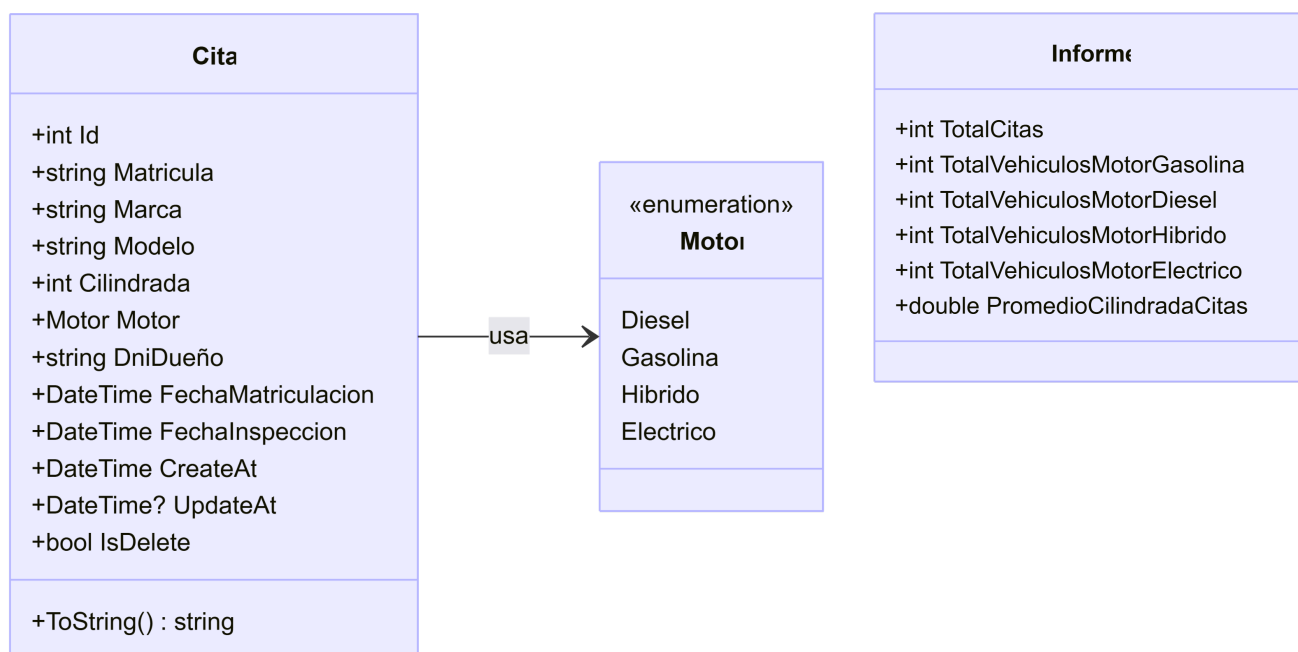


Diagrama de casos de uso

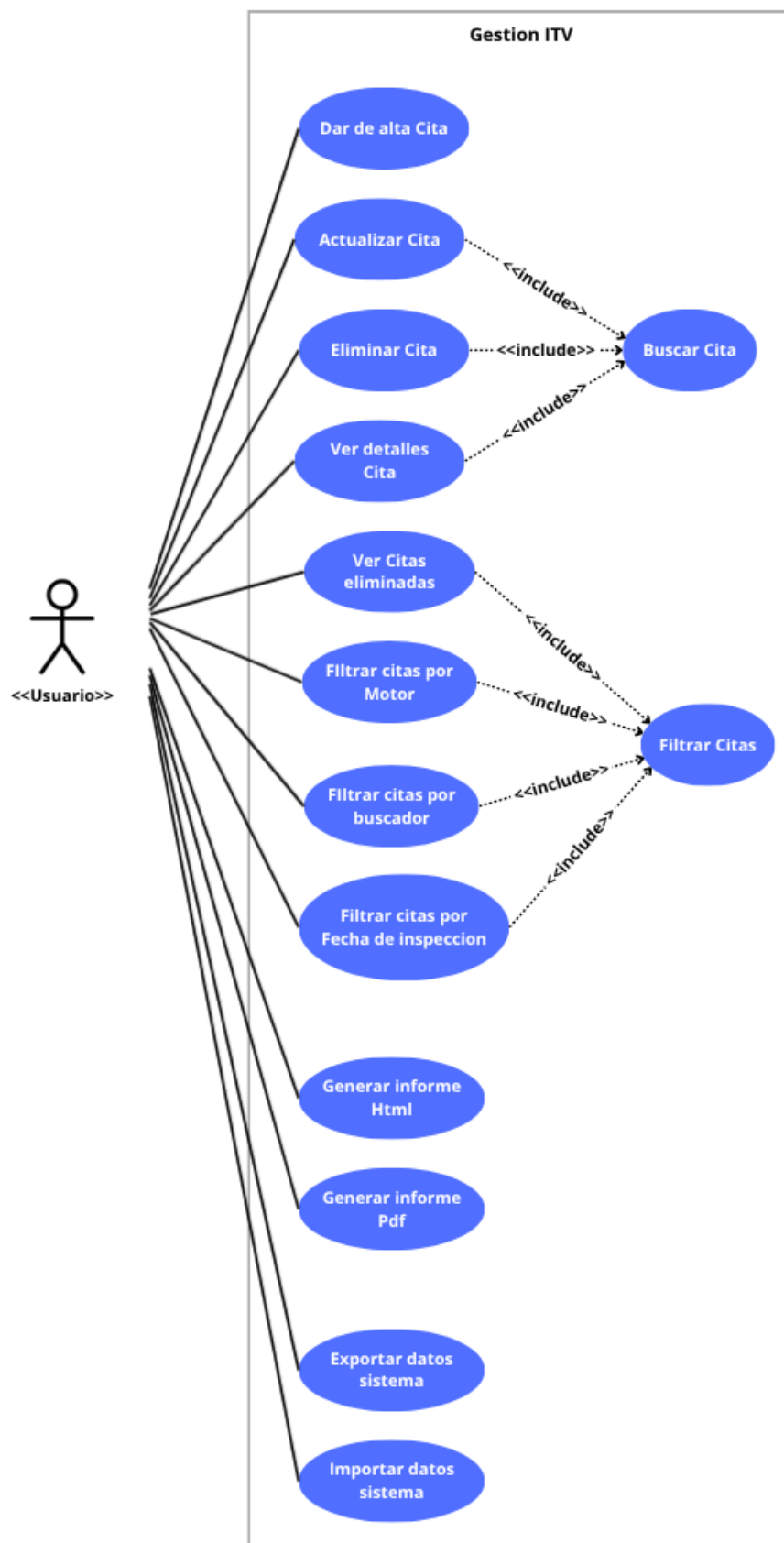


Diagrama de secuencia: Create

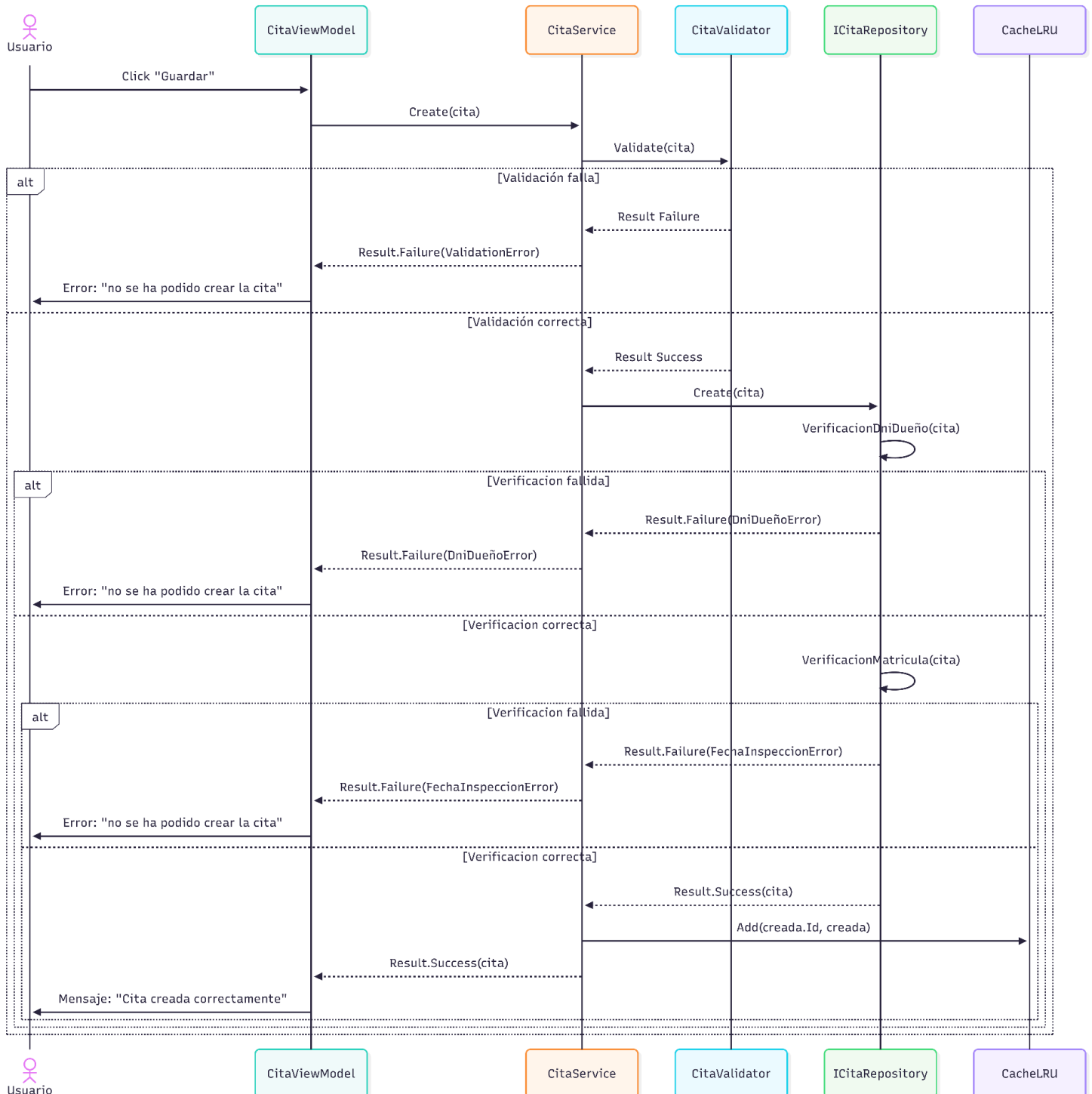


Diagrama de secuencia: Update

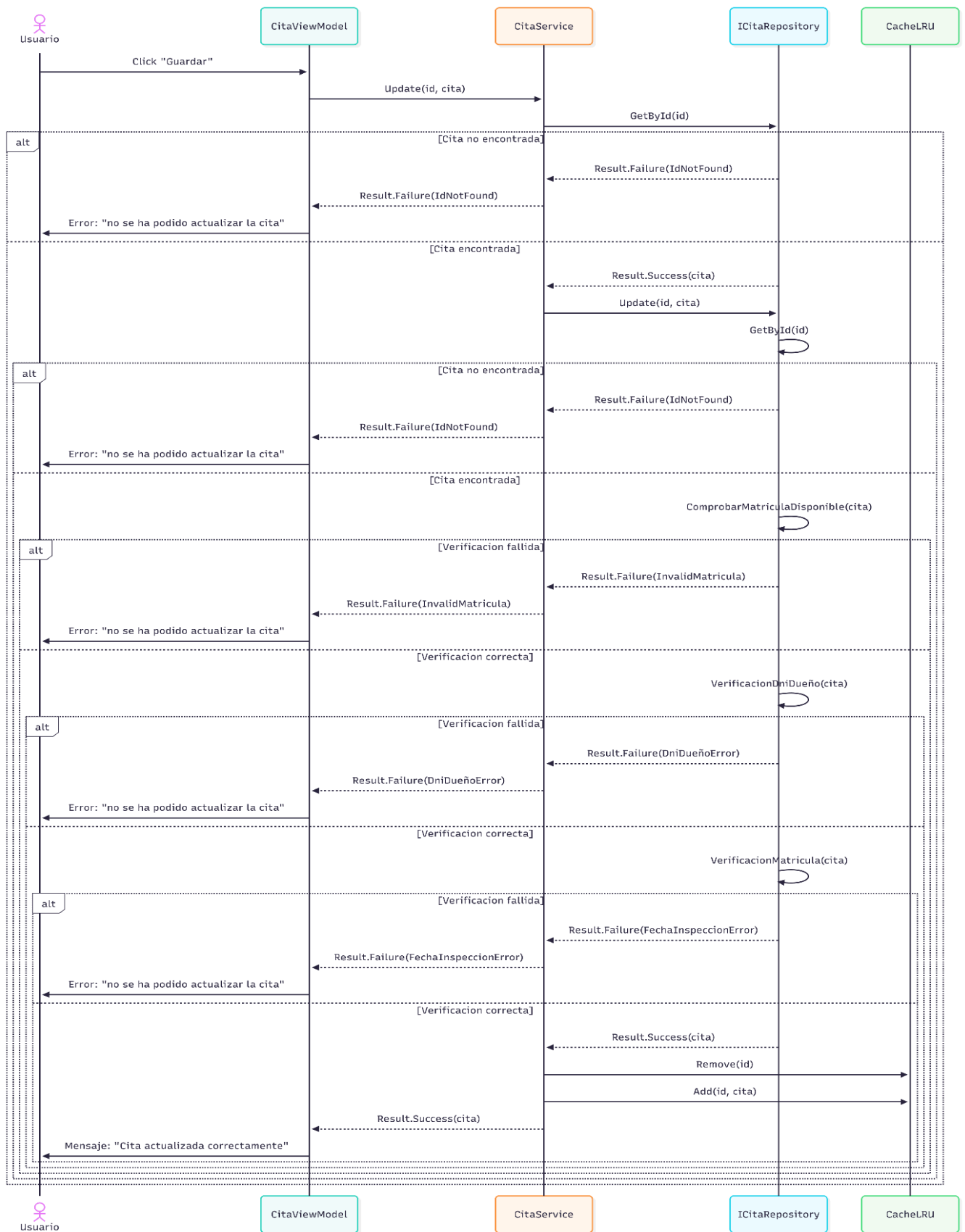


Diagrama de secuencia: Delete

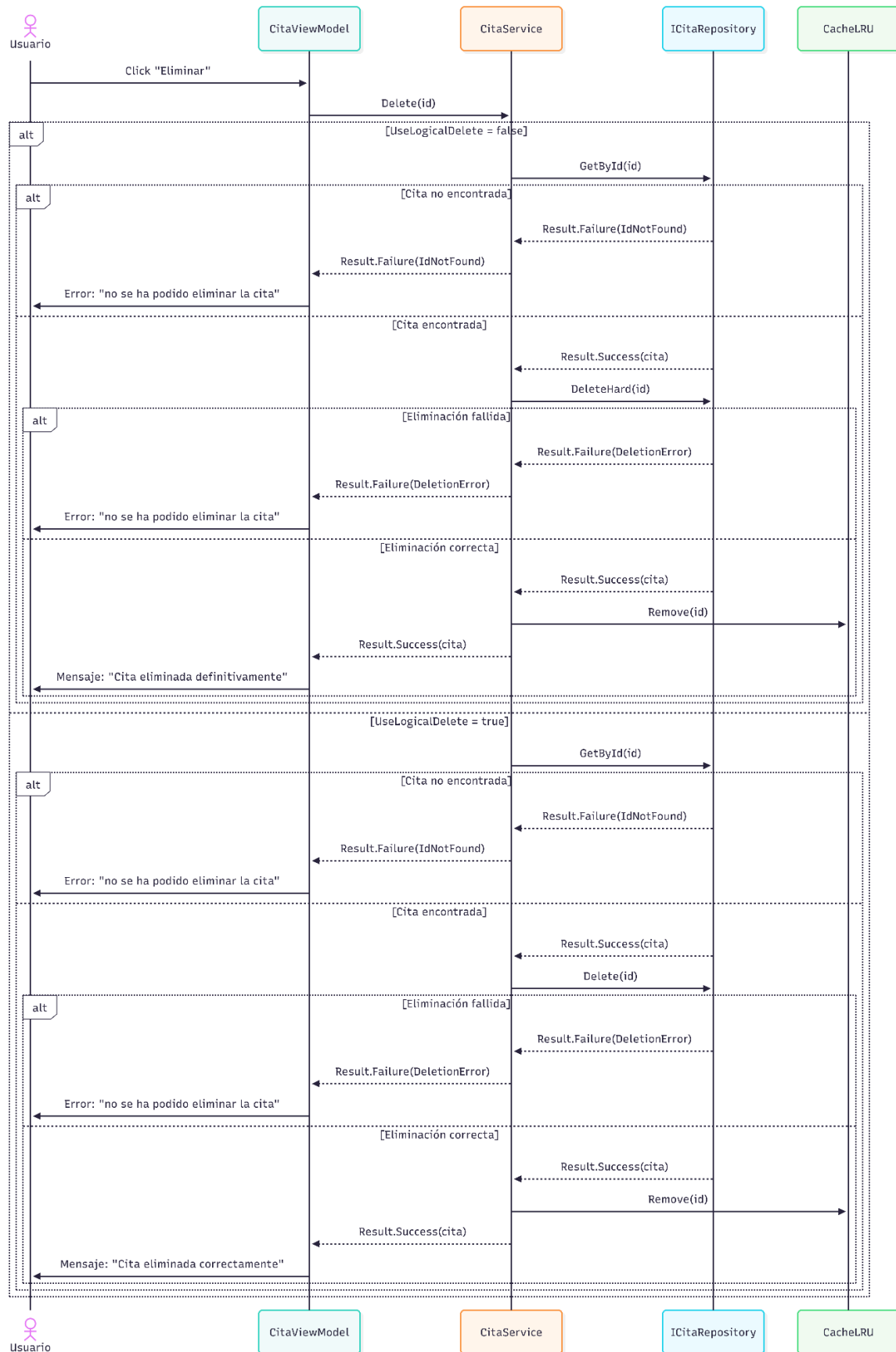


Diagrama de secuencia: GetAll

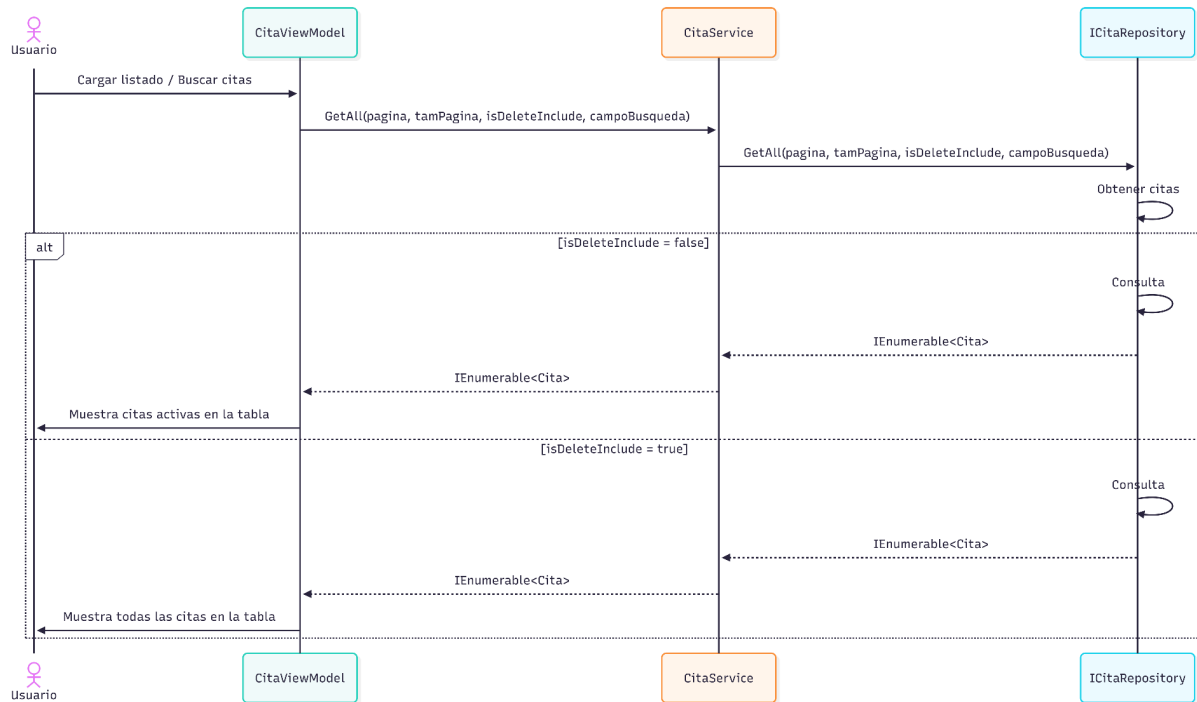
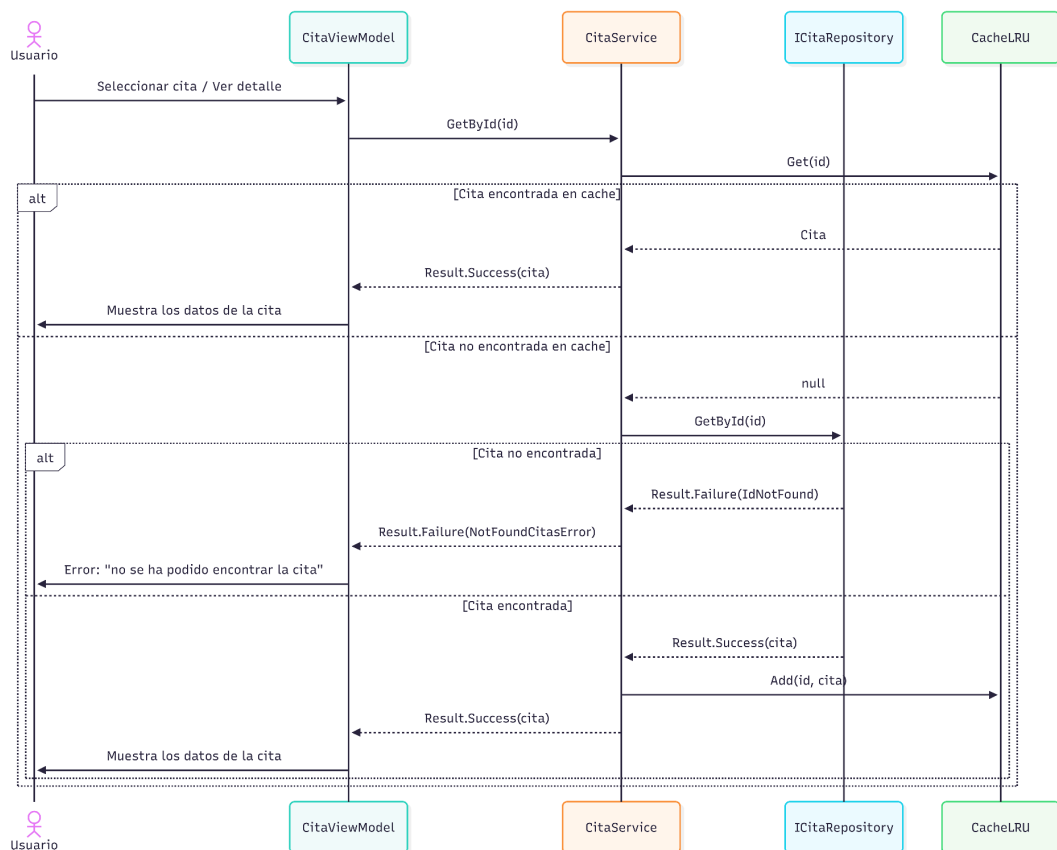


Diagrama de secuencia: GetById

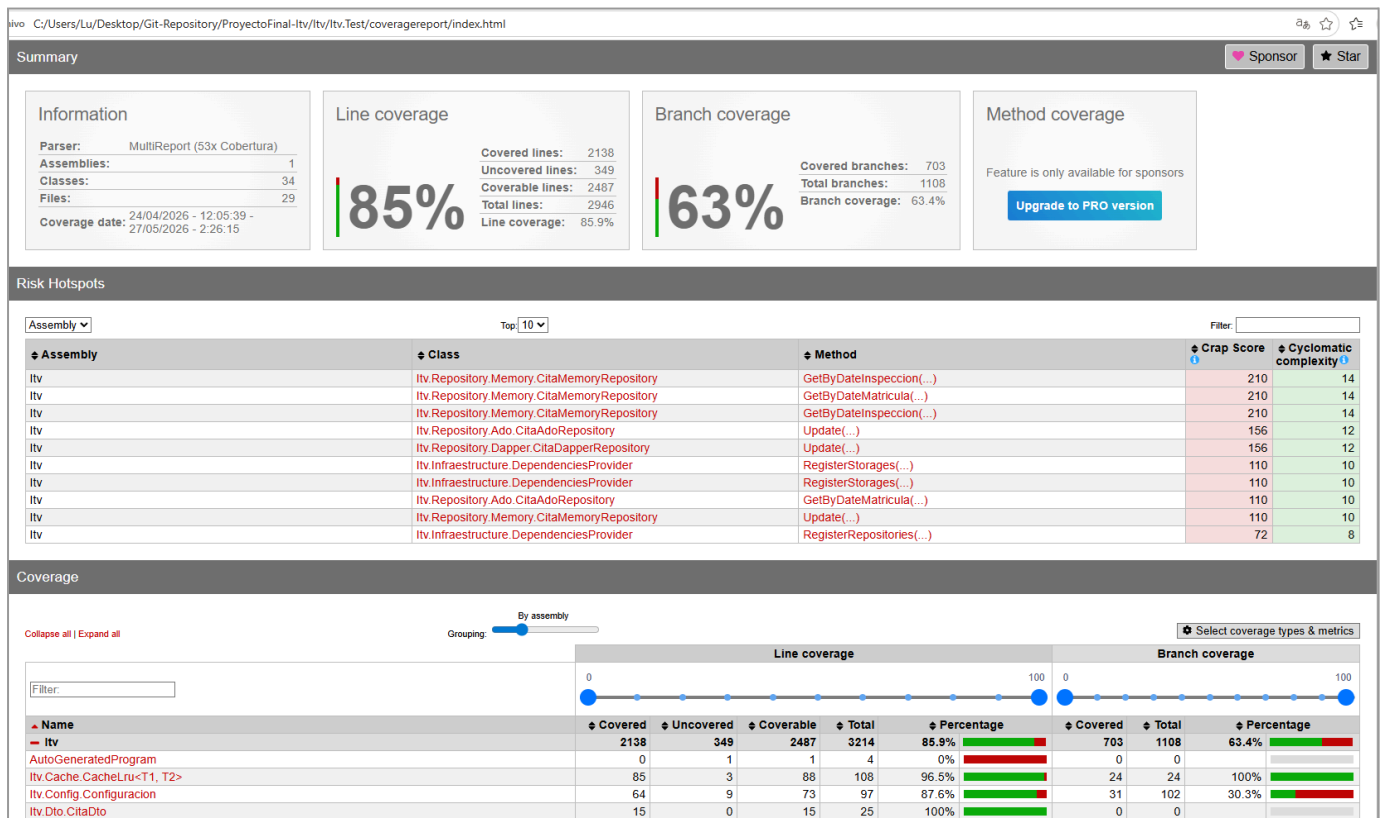


Diseño de bases de datos

CITA		
int	Id	PK
string	Matricula	
string	Marca	
string	Modelo	
int	Cilindrada	
string	Motor	
string	DniDueño	
string	FechaMatriculacion	
string	FechaInspeccion	
string	CreateAt	
string	UpdateAt	
bool	IsDelete	

Cobertura de pruebas

Para garantizar la calidad y el correcto funcionamiento del sistema se han realizados test unitarios sobre la capa de la lógica de negocio, estas pruebas ayudan a comprobar el correcto funcionamiento de los servicios, repositorios, validadores, cache, storage, y los informes, en sus respectivos test unitarios se comprueban tanto casos válidos como casos inválidos para intentar llegar al máximo porcentaje de cobertura de código.



Estimacion de analisis economico

Se ha calculado una estimación económica del proyecto, para la que se han tenido en cuenta los costes de las herramientas utilizadas y la dedicación de horas para cada fase del proyecto, las cuales son: análisis, desarrollo, pruebas, documentación y gestión.

Bloque	Concepto / Fase	Coste unitario	Cantidad	Coste total
Infraestructura	Licencia JetBrains Rider	300 € / año	1	300 €
Subtotal infraestructura				300 €
Desarrollo	Análisis de requisitos y diseño	20 € / h	25 h	500 €
Desarrollo	Capa de servicios y lógica de negocio	20 € / h	40 h	800 €
Desarrollo	Implementación de repositorios ADO.NET, Dapper, EFC y Memory	20 € / h	130 h	2.600 €
Desarrollo	Implementación frontend WPF y MVVM	20 € / h	60 h	1.200 €
Desarrollo	Importación, exportación e informes	20 € / h	35 h	700 €
Desarrollo	Testing y control de calidad	20 € / h	120 h	2.400 €
Desarrollo	Documentación técnica y funcional	20 € / h	25 h	500 €
Desarrollo	Gestión y seguimiento del proyecto	20 € / h	15 h	300 €
Subtotal desarrollo			450 h	9.000 €